

Informe de Desempeno MPI Document Search Engine

Proyecto Final

1 Resumen

Este proyecto implementa un motor de busqueda paralelo con MPI basado en TF-IDF y similitud coseno. En esta version la indexacion se distribuye entre los procesos (TF-IDF e IDF en paralelo) y se reduce el volumen de datos transferidos al intercambiar solo el vocabulario global y los Top-K, lo cual mejora el rendimiento global. El informe presenta el algoritmo, la paralelizacion y un analisis detallado con las graficas obtenidas.

2 Conceptos clave (explicados en simple)

- **Corpus:** conjunto de documentos de texto usados como base de busqueda.
- **Vocabulario:** lista de palabras unicas presentes en el corpus.
- **Tokenizacion:** proceso de dividir el texto en palabras (tokens).
- **TF (Term Frequency):** frecuencia relativa de una palabra en un documento.
- **DF (Document Frequency):** numero de documentos que contienen una palabra.
- **IDF (Inverse Document Frequency):** peso que reduce palabras muy comunes.
- **TF-IDF:** producto $TF \times IDF$; representa importancia de una palabra.
- **Similitud coseno:** mide el angulo entre dos vectores; 1 significa muy similares.
- **Top-K:** los K documentos con mayor score de similitud.
- **Scatter/Gather/Bcast/Allreduce:** operaciones MPI para repartir, recolectar, difundir y reducir datos entre procesos.
- **Rank:** identificador de cada proceso MPI (0 es el master).

3 Generacion de corpus de prueba

Los corpus usados en las pruebas se generan con `generate_corpus.py`. El script crea un vocabulario sintetico de tamano configurable (por defecto 5000), genera documentos con una distribucion tipo Zipf (algunas palabras aparecen mucho mas que otras) y produce una consulta con un numero fijo de terminos. Esto permite crear conjuntos de 100, 500, 1000 y 5000 documentos de manera reproducible usando una semilla fija.

4 Descripcion del algoritmo

Cada documento se convierte en un vector TF-IDF y la consulta tambien se convierte en un vector del mismo tamano. Luego se calcula la similitud coseno entre la consulta y cada documento. Finalmente se ordenan los resultados y se devuelven los Top-K documentos mas relevantes.

4.1 TF-IDF

Para cada termino t en el documento d :

$$TF(t, d) = \frac{\text{freq}(t, d)}{\text{total de terminos en } d}$$

$$IDF(t) = \log \left(\frac{N}{1 + df(t)} \right)$$

El vector final es $TF-IDF(t, d) = TF(t, d) \cdot IDF(t)$, donde N es el numero total de documentos y $df(t)$ es el numero de documentos que contienen el termino.

4.2 Similitud coseno

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

Los documentos se ordenan de mayor a menor segun esta similitud.

5 Paralelizacion con MPI

La version actual distribuye la indexacion y reduce comunicacion:

1. **Particion del corpus:** todos los ranks listan los archivos y cargan solo su subconjunto local.
2. **Vocabulario global:** cada rank crea un vocabulario local; el rank 0 los fusiona y lo difunde con `MPI_Bcast`.
3. **DF/IDF en paralelo:** cada rank calcula DF local y se combina con `MPI_Allreduce` para obtener DF global e IDF.
4. **TF-IDF local:** cada rank calcula TF-IDF solo para sus documentos.
5. **Top-K distribuido:** cada rank calcula similitudes y envia su Top-K; el master los fusiona con `MPI_Gatherv`.

Este flujo elimina el `Scatter` de la matriz TF-IDF completa y reduce el trafico total a vocabulario y Top-K.

6 Pipeline del search_engine

El flujo completo del programa puede resumirse en los siguientes pasos:

1. **Lectura del corpus:** cada rank lista los archivos y carga solo su particion local en memoria.
2. **Vocabulario global:** se construye un vocabulario local y se fusiona en el master para producir el vocabulario global.

3. **Calculo de DF/IDF:** cada rank calcula DF local, se reduce para obtener DF global y se calcula IDF.
4. **Vectorizacion:** cada rank calcula TF-IDF para sus documentos y el vector TF-IDF de la consulta.
5. **Busqueda:** se calcula la similitud coseno para cada documento local y se obtiene el Top-K local.
6. **Recoleccion final:** el master recibe los Top-K locales, realiza el merge y entrega el ranking final.

Este pipeline esta alineado con la forma en la que se midieron los tiempos de indexacion, busqueda y merge en los resultados experimentales.

7 Mejora de rendimiento: TF-IDF/IDF en paralelo y menos datos transferidos

La ultima mejora se enfoco en reducir el tiempo de indexacion y el trafico MPI. Antes, el master calculaba el vocabulario, IDF y toda la matriz TF-IDF, y luego distribuia esa matriz completa con `MPI_Scatterv`. Esto generaba dos problemas: (1) una fase de indexacion totalmente serial en el master y (2) un volumen grande de datos enviados a todos los procesos.

En la version actual, el trabajo se distribuye desde el inicio:

- **Vocabulario paralelo:** cada rank construye su vocabulario local y el master los fusiona para crear el vocabulario global.
- **DF/IDF paralelos:** cada rank calcula DF local y se realiza un `MPI_Allreduce` para obtener DF global y calcular IDF en todos.
- **TF-IDF local:** cada rank calcula TF-IDF solo para sus documentos, eliminando la necesidad de enviar una matriz completa por la red.
- **Comunicacion reducida:** solo se transfieren el vocabulario global y los Top-K locales, lo que reduce significativamente el trafico MPI.

El impacto se observa en los resultados: el tiempo de **Scatter** es practicamente cero (Figura 4) y el tiempo total baja con P en corpus grandes (Figura 1), con speedup notable hasta 32 procesos (Figura 2). Con el merge jerarquico, el costo de recoleccion de Top-K se reduce, aunque sigue siendo el principal limite en P muy altos.

8 Mejora de rendimiento: merge jerarquico de Top-K

Para reducir el costo del **Gather**, se reemplazo la recoleccion plana de resultados por un **merge jerarquico** (reduccion en arbol). En lugar de enviar todos los Top-K directamente al master, los procesos se emparejan y fusionan sus listas de manera progresiva:

- Cada rank parte con su Top-K local ordenado.
- En cada nivel, ranks pares reciben el Top-K de su pareja, fusionan y conservan solo los mejores K resultados.

- El numero de mensajes se reduce a $\log_2(P)$ niveles y el volumen total transferido disminuye, ya que cada fusion mantiene solo K elementos.

Este esquema reduce la presion sobre el master y baja el tiempo de merge en configuraciones con muchos procesos, aunque el overhead de sincronizacion todavia limita el escalamiento para P muy altos.

9 Configuracion experimental

Las pruebas se realizaron en el cluster usando los mismos parametros que el estudio anterior. Se evaluaron corpus de 100, 500, 1000 y 5000 documentos, con vocabulario de alrededor de 5000 palabras, Top-K = 10 y multiples repeticiones. Los resultados se almacenan en **results/**.

9.1 Configuracion de ejecucion en el cluster

La ejecucion en el cluster se definio mediante un script SLURM **run_search.sh** con los siguientes parametros principales:

- **Particion:** legacy
- **Tiempo maximo:** 04:00:00
- **Nodos:** 4
- **Tareas por nodo:** 16 (hasta 64 procesos en total)
- **Memoria por CPU:** 512 MB
- **MPI:** OpenMPI GCC 4.1.6 (modulo **mpi/openmpi-gcc11/4.1.6**)

Esta configuracion permite ejecutar los experimentos con $P = 1, 2, 4, 8, 16, 32$ y 64 procesos, manteniendo coherencia con las pruebas de escalabilidad reportadas en este informe.

10 Resultados y analisis

10.1 Tiempo total

La Figura 1 muestra reducciones claras del tiempo total al aumentar procesos, especialmente para corpus grandes. Por ejemplo, para 5000 documentos el tiempo total baja de 46.1 s ($P=1$) a 11.5 s ($P=4$) y alcanza un minimo de 8.37 s en $P=32$; en $P=64$ vuelve a crecer por sobrecarga. En corpus pequenos el overhead limita la mejora.

10.2 Speedup y eficiencia

La Figura 2 evidencia mejoras sustanciales: para 5000 documentos el speedup llega a 4.02 ($P=4$) y 5.50 ($P=32$). Para 1000 documentos el mejor speedup es 3.35 ($P=4$). La eficiencia es alta en $P=2-4$ para corpus grandes (cercana a 1), pero disminuye al aumentar P por sobrecarga de sincronizacion y recoleccion de resultados.

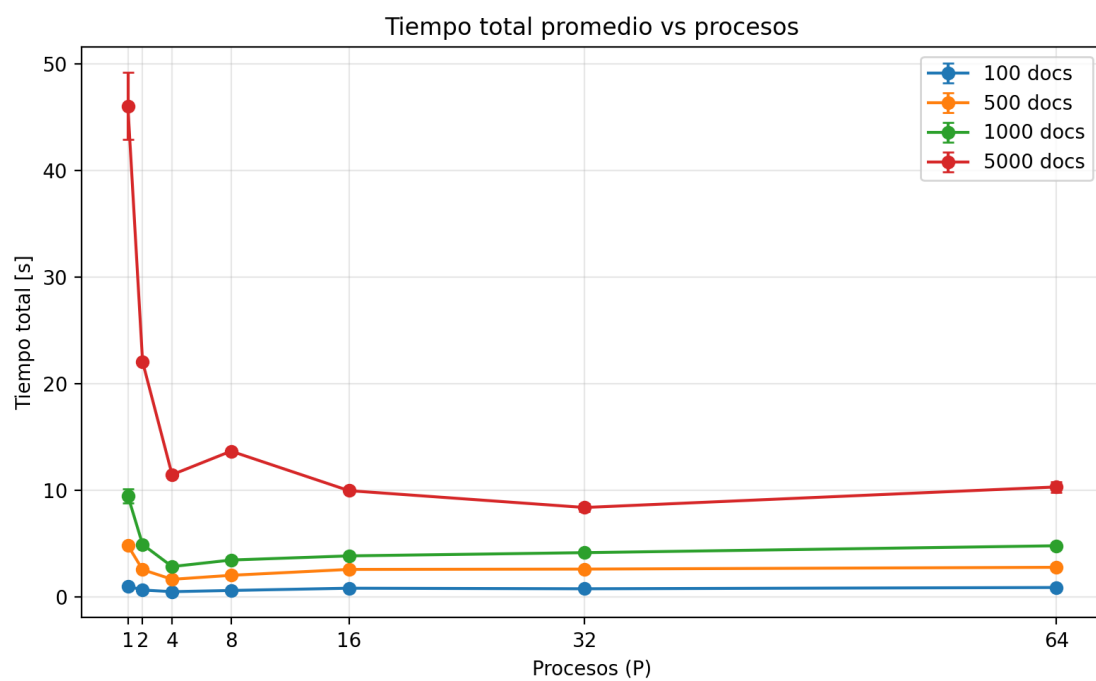


Figure 1: Tiempo total promedio vs procesos.

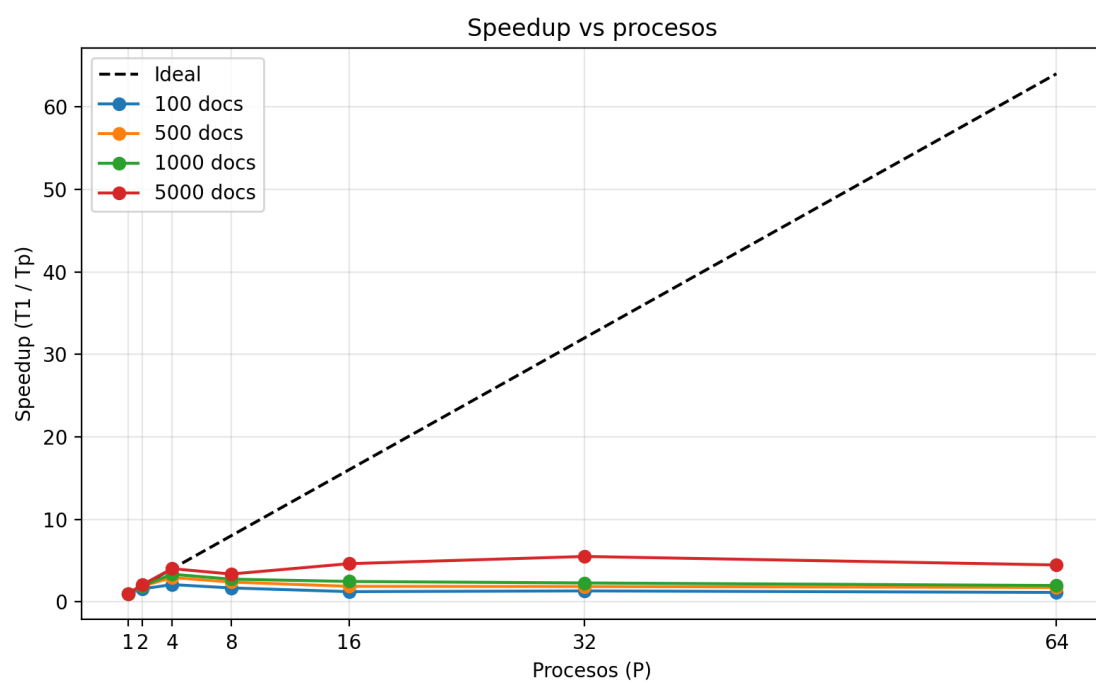


Figure 2: Speedup por cantidad de procesos.

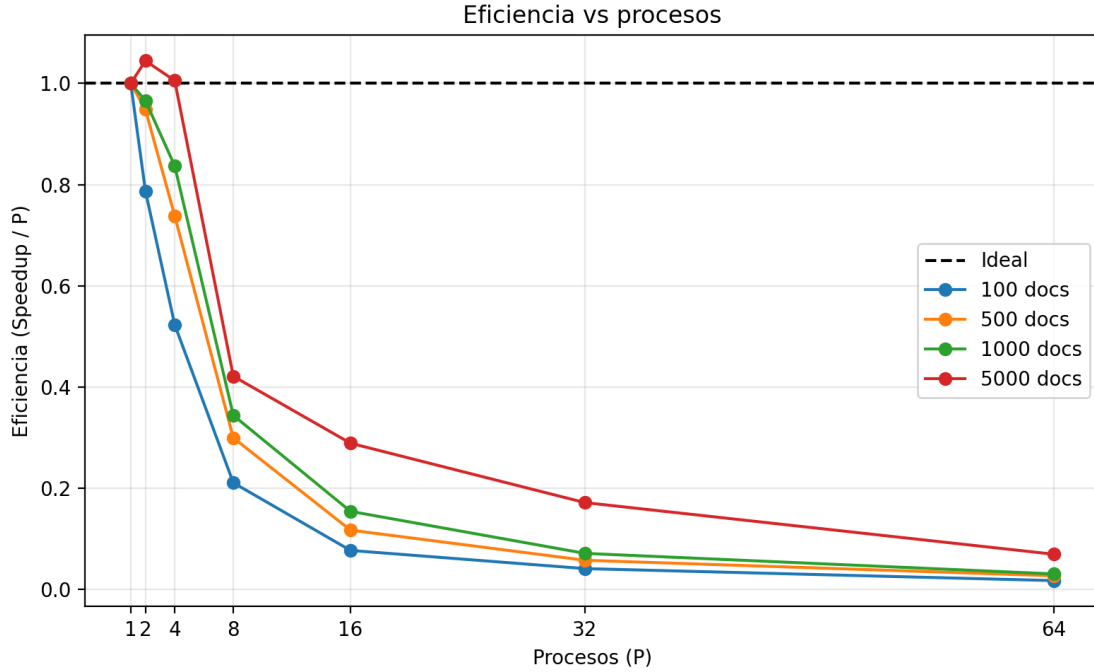


Figure 3: Eficiencia por cantidad de procesos.

10.3 Desglose por fases

La Figura 4 muestra que **Scatter** desaparece (tiempo aproximadamente cero) al eliminar la distribución de la matriz TF-IDF. La indexación ahora se reparte entre procesos y disminuye con P. Con el merge jerárquico, el costo de **Gather/Merge** baja en P altos, pero sigue creciendo con el número de procesos y domina el overhead en escalas grandes.

10.4 Observaciones clave

- **Mejora global:** la indexación paralela reduce el tiempo total de forma notable, especialmente en corpus grandes.
- **Overhead en P altos:** el merge jerárquico reduce el costo del **Gather/Merge**, pero aun limita la eficiencia cuando P es muy grande.
- **Mejor punto operativo:** para 5000 documentos, P=16–32 ofrece el mejor balance entre speedup y eficiencia.

11 Conclusiones

La versión paralelizada de TF-IDF e IDF mejora claramente el rendimiento y reduce el tráfico MPI, eliminando el **Scatter** de matrices grandes. El merge jerárquico disminuye el costo de recolección de Top-K, pero en P muy altos el overhead de fusión y sincronización sigue siendo el principal límite.

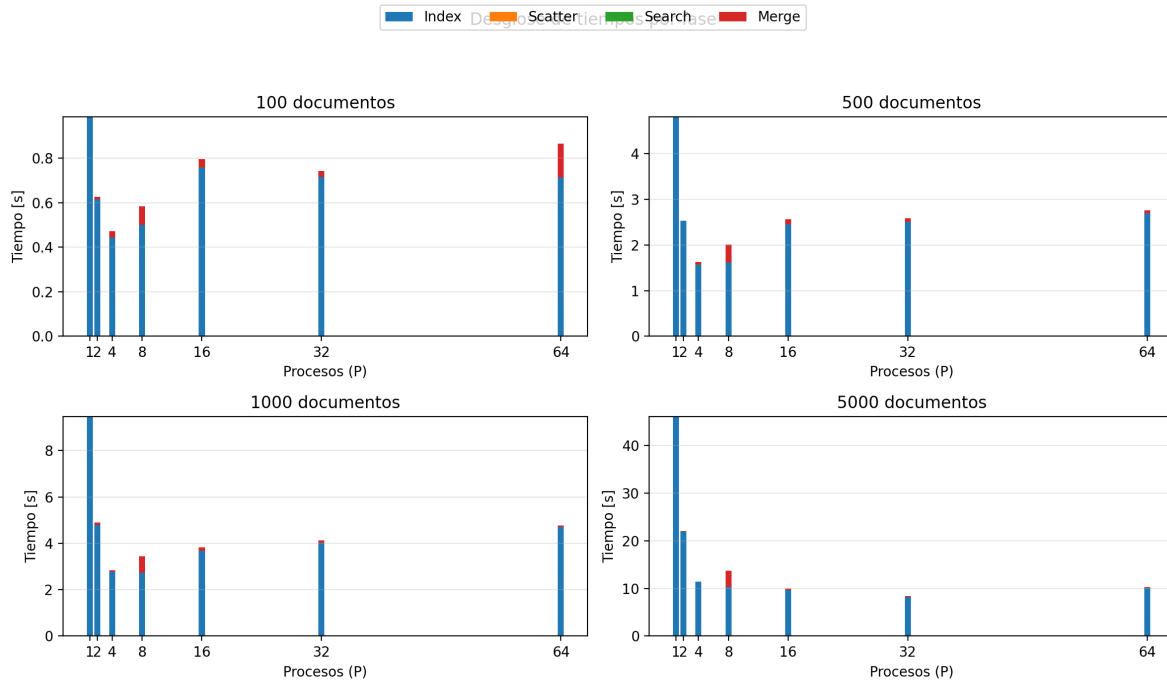


Figure 4: Desglose de tiempos por fase (Index/Scatter/Search/Merge).

12 Como reproducir las graficas

Ejecutar los scripts desde la raiz del proyecto:

```
python3 plot_scaling.py
python3 plot_breakdown.py
```

Las figuras se guardan en el directorio **figuras/**.